

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR
EMSI

[LOGO EMSI]

RAPPORT DE PROJET
MODULE : DEEP LEARNING

Implémentation et Analyse de Modèles de Deep Learning avec PyTorch

Classification d'Images (Fashion-MNIST) et Analyse de Sentiments (IMDb)

Rédigé par :
Elfarouki Abderrahmane
Élève Ingénieur

Encadré par :
M.BATTAL
Enseignant - EMSI

Année Universitaire : 2025 – 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à mon professeur et encadrant pour son orientation, ses précieux conseils et son soutien tout au long de ce projet de Deep Learning.

Mes remerciements s'adressent également à l'administration et au corps enseignant de l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour avoir mis à notre disposition les ressources pédagogiques nécessaires à la réussite de ce travail.

Enfin, je remercie toute personne ayant contribué de près ou de loin à l'élaboration de ce projet.

Résumé

Ce projet présente la conception, l'implémentation et l'évaluation de deux architectures de réseaux de neurones profonds en utilisant le framework PyTorch.

La première partie est dédiée à la vision par ordinateur, où un réseau de neurones convolutif (CNN), nommé **FashionCNN**, est développé pour classifier les articles de mode du jeu de données **Fashion-MNIST**. Ce modèle est composé de deux couches convolutives suivies de couches de Max Pooling, d'un Dropout et de couches denses.

La seconde partie traite de l'analyse de sentiments dans le domaine du traitement du langage naturel (NLP). Un modèle d'embedding continu avec pooling moyen (**SentimentModel**) est entraîné sur le jeu de données ****IMDb Movie Reviews**** pour classer des critiques de films comme positives ou négatives.

Ce rapport détaille la méthodologie adoptée, le prétraitement des données, les architectures choisies, les expérimentations menées et une analyse rigoureuse des résultats obtenus, illustrée par les courbes de perte (loss) et les visualisations d'erreurs réelles issues de l'exécution du projet.

Table des matières

Remerciements	1
Résumé	2
1 Introduction	4
1.1 Contexte	4
1.2 Objectifs du Projet	4
2 Présentation du Problème et Données	5
2.1 Classification d'Images (Fashion-MNIST)	5
2.1.1 Description du Dataset	5
2.2 Analyse de Sentiments (IMDb)	5
2.2.1 Description du Dataset	5
3 Méthodologie et Implémentation	7
3.1 Prétraitement des Données	7
3.1.1 Données Fashion-MNIST	7
3.1.2 Données IMDb	7
3.2 Architectures des Modèles	7
3.2.1 Modèle de Vision : FashionCNN	7
3.2.2 Modèle NLP : SentimentModel	8
3.3 Environnement de Développement et Outils	8
4 Expérimentations et Résultats	9
4.1 Entraînement des Modèles	9
4.2 Résultats Obtenus	9
4.2.1 Classification Fashion-MNIST	9
4.2.2 Analyse de Sentiments IMDb	9
5 Analyse et Discussion	11
5.1 Courbes de Perte (Loss)	11
5.2 Visualisation et Analyse des Erreurs (FashionCNN)	11
5.3 Limites du Modèle d'Analyse de Sentiment	11
6 Difficultés Rencontrées et Conclusion	13
6.1 Difficultés Rencontrées	13
6.2 Conclusion et Perspectives	13

Chapitre 1

Introduction

1.1 Contexte

Dans le domaine de l'intelligence artificielle, le Deep Learning a révolutionné la façon dont les machines traitent les données complexes comme les images et le texte. Ce projet s'inscrit dans le cadre de notre formation d'ingénieur à l'EMSI, visant à acquérir une maîtrise pratique du framework PyTorch, l'un des standards industriels et académiques pour la conception de réseaux de neurones.

1.2 Objectifs du Projet

Ce projet vise à résoudre deux problématiques représentatives de l'apprentissage profond :

1. **La classification multiclasse d'images** : Identifier automatiquement le type de vêtement représenté sur une image en niveaux de gris (10 classes possibles).
2. **La classification binaire de textes** : Analyser une critique de film textuelle afin de prédire si l'opinion émise est positive ou négative.

Le projet privilégie une approche modulaire et structurée pour assurer la maintenabilité et la clarté du code source.

Chapitre 2

Présentation du Problème et Données

2.1 Classification d’Images (Fashion-MNIST)

Le premier problème consiste à classifier des images de vêtements de 28×28 pixels. Contrairement au traditionnel dataset MNIST (chiffres manuscrits), Fashion-MNIST présente une complexité accrue en raison de la diversité des textures et des motifs des articles.

2.1.1 Description du Dataset

Fashion-MNIST contient 70 000 exemples au total :

- **Jeu d’entraînement** : 60 000 images.
- **Jeu de test** : 10 000 images.

Chaque image est étiquetée parmi les 10 classes suivantes :

TABLE 2.1 – Classes du jeu de données Fashion-MNIST

Index	Classe	Index	Classe
0	T-shirt	5	Sandale
1	Pantalon	6	Chemise
2	Pull	7	Sneaker
3	Robe	8	Sac
4	Manteau	9	Bottine

2.2 Analyse de Sentiments (IMDb)

Le deuxième problème est une tâche de classification de texte binaire. Il s’agit d’analyser le sentiment général d’une critique de film.

2.2.1 Description du Dataset

Le dataset IMDb Movie Reviews comporte 50 000 critiques de films hautement polarisées récupérées sur le site IMDb :

- **Jeu d’entraînement** : 25 000 critiques.
- **Jeu de test** : 25 000 critiques.

Les deux classes sont équitablement réparties (50% de critiques positives et 50% de critiques négatives).

Chapitre 3

Méthodologie et Implémentation

Le projet est structuré de manière modulaire afin d'isoler les modèles, le chargement des données et les routines d'entraînement.

3.1 Prétraitement des Données

3.1.1 Données Fashion-MNIST

Les images brutes subissent une chaîne de transformations :

- Conversion en tenseur PyTorch via `transforms.ToTensor()`, normalisant les intensités de pixels de $[0, 255]$ à $[0.0, 1.0]$.
- Normalisation avec une moyenne de 0.5 et un écart-type de 0.5 via `transforms.Normalize((0.5, 0.5, ),)`, recentrant les valeurs autour de $[-1.0, 1.0]$.

3.1.2 Données IMDB

Pour les textes IMDB, un prétraitement spécifique est codé dans `SentimentDataset` :

1. Le texte est converti en minuscules et découpé en mots simples (tokenisation par espace).
2. Une troncature à une longueur fixe $L_{max} = 50$ mots est appliquée.
3. Les mots sont convertis en indices entiers grâce à un vocabulaire pré-construit des 10 000 mots les plus fréquents. Les mots absents reçoivent l'index 1 (<UNK>).
4. Un rembourrage (*padding*) à base de zéros (<PAD>) est ajouté pour atteindre la longueur fixe de 50.

3.2 Architectures des Modèles

3.2.1 Modèle de Vision : FashionCNN

Défini dans `models/model.py`, le modèle utilise l'architecture suivante :

- **Conv2D 1** : Canal d'entrée = 1, Canaux de sortie = 32, Noyau = 3×3 , Padding = 1.
- **ReLU** suivie de **MaxPool2D** (2×2 , stride = 2) $\rightarrow$ Taille de sortie : $32 \times 14 \times 14$.
- **Conv2D 2** : Canaux d'entrée = 32, Canaux de sortie = 64, Noyau = 3×3 , Padding = 1.

- **ReLU** suivie de **MaxPool2D** (2×2 , stride = 2) $\rightarrow$ Taille de sortie : $64 \times 7 \times 7$.
- **Flattening** : Transformation du tenseur $64 \times 7 \times 7$ en un vecteur unidimensionnel de taille 3136.
- **Linear 1** : Taille d'entrée = 3136, Taille de sortie = 128, suivie d'une fonction d'activation ReLU.
- **Dropout** : Taux de 25% pour limiter le surapprentissage.
- **Linear 2 (Sortie)** : Taille d'entrée = 128, Taille de sortie = 10 (une pour chaque classe).

3.2.2 Modèle NLP : SentimentModel

Défini dans `models/sentiment_model.py`, le réseau est structuré ainsi :

Embedding Layer : Taille du vocabulaire $\times$ 64 dimensions. Traduit chaque mot en un vecteur continu. Le padding à l'index 0 est ignoré.

Global Average Pooling : Calcule la moyenne des embeddings sur l'axe de la séquence de texte pour obtenir un vecteur unique représentatif du texte.

Linear Layer : Taille d'entrée = 64, Taille de sortie = 1 (valeur brute pré-sigmoïdienne).

3.3 Environnement de Développement et Outils

- **Langage** : Python.
- **Framework Principal** : PyTorch (gestion des tenseurs, rétropropagation, couches neuronales).
- **Visualisation** : Matplotlib (tracé des courbes de loss et visualisation d'images).
- **NLP** : Hugging Face `datasets` pour le téléchargement robuste de IMDb.
- **Métriques** : Scikit-learn (pour le calcul du rapport de classification).

Chapitre 4

Expérimentations et Résultats

4.1 Entraînement des Modèles

Les deux modèles ont été entraînés sur 5 époques avec l'optimiseur **Adam** (taux d'apprentissage $\eta = 0.001$).

- Pour **FashionCNN**, la fonction de perte utilisée est **CrossEntropyLoss**.
- Pour **SentimentModel**, la fonction de perte est **BCEWithLogitsLoss**.

4.2 Résultats Obtenus

4.2.1 Classification Fashion-MNIST

L'exécution de la boucle d'entraînement affiche la diminution progressive de la perte par époque.

TABLE 4.1 – Pertes enregistrées pendant l'entraînement de FashionCNN

Époque	Somme des pertes (Loss cumulée)
1	392.2036
2	260.4079
3	220.8927
4	195.9680
5	175.7602

À l'issue de l'entraînement, l'évaluation du modèle sur le jeu de test affiche les performances suivantes :

- **Précision (Accuracy)** : 91.13%

4.2.2 Analyse de Sentiments IMDb

Le tableau suivant résume la diminution de la loss du modèle d'embedding sur 5 époques.

TABLE 4.2 – Pertes enregistrées pendant l’entraînement de SentimentModel

Époque	Somme des pertes (Loss cumulée)
1	265.3129
2	232.0837
3	207.9863
4	191.1311
5	178.6923

L’évaluation finale sur le jeu de test fournit le rapport de classification suivant :

TABLE 4.3 – Rapport de performance - Sentiment IMDB

Classe	Précision	Rappel	F1-Score	Support
Négatif	0.77	0.79	0.78	12500
Positif	0.78	0.77	0.77	12500
Global (Accuracy)			0.78	25000

Chapitre 5

Analyse et Discussion

5.1 Courbes de Perte (Loss)

Les figures générées lors de l'exécution montrent la convergence des modèles.

FIGURE 5.1 – Courbe de perte (Loss) de FashionCNN

La courbe décroît régulièrement, indiquant un apprentissage stable sans divergence majeure sur les 5 époques programmées.

FIGURE 5.2 – Courbe de perte (Loss) de SentimentModel sur IMDB

5.2 Visualisation et Analyse des Erreurs (FashionCNN)

Le script extrait les prédictions erronées sur l'ensemble de test et les enregistre dans `erreurs_fashion.png`.

FIGURE 5.3 – Exemple de 10 prédictions incorrectes de FashionCNN

L'analyse de ces images montre que la plupart des erreurs proviennent de confusions logiques entre des catégories visuellement très similaires :

- Confusion récurrente entre les classes **Pull** (Pullover), **Manteau** (Coat) et **Chemise** (Shirt).
- Confusion entre les chaussures : **Sandale**, **Sneaker** et **Bottine**.

Ces erreurs sont partagées même par des observateurs humains sur des images basse résolution en niveaux de gris.

5.3 Limites du Modèle d'Analyse de Sentiment

L'accuracy globale plafonne à 78%. Cette performance modeste s'explique par les limitations suivantes :

1. **Perte d'informations temporelles** : Le modèle fait la moyenne des embeddings. Deux phrases avec les mêmes mots dans un ordre différent auront la même signature d'embedding.
2. **Taille fixe trop courte** : Fixer la longueur des textes à 50 mots tronque la majorité des avis IMDb, perdant ainsi le contexte de fin de critique.
3. **Tokenisation rudimentaire** : L'absence de nettoyage de la ponctuation fragmente inutilement le dictionnaire (ex : "super!" et "super" comptent pour deux mots distincts).

Chapitre 6

Difficultés Rencontrées et Conclusion

6.1 Difficultés Rencontrées

- **Problème de bibliothèque doublée (KMP Duplicate Lib)** : Une erreur liée à l'environnement d'exécution OpenMP sous Windows a nécessité l'utilisation de la variable d'environnement `os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"`.
- **Duplication de code** : Les versions obsolètes des codes d'entraînement et d'évaluation dans les fichiers de départ rendaient le débogage complexe (absence de `model.eval()` et de `model.train()` dans certaines branches).
- **Limitations matérielles** : L'exécution du jeu de données IMDB sur CPU ralentit grandement la vitesse d'apprentissage de l'embedding.

6.2 Conclusion et Perspectives

Ce projet a permis de mettre en œuvre deux pipelines complets de Deep Learning avec PyTorch. Le modèle CNN a montré une très bonne efficacité pour le traitement d'images (91.13% d'accuracy), tandis que le modèle d'embedding sur le texte a posé les bases de la classification en NLP.

Pour de futures itérations, nous envisageons :

- L'intégration d'un réseau récurrent (LSTM/GRU) ou d'un CNN 1D pour l'analyse de texte.
- La mise en œuvre d'un script de détection automatique pour exploiter l'accélération matérielle GPU (CUDA).

Bibliographie

- [1] Adam Paszke, Sam Gross, Francisco Massa, et al. *PyTorch : An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019).
- [2] Han Xiao, Kashif Rasul, Roland Vollgraf. *Fashion-MNIST : a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv :1708.07747, 2017.
- [3] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. *Learning Word Vectors for Sentiment Analysis*. Association for Computational Linguistics (ACL), 2011.